

Introduction to Regression

* Regression **is** a supervised machine learning technique used to predict a continuous numerical value based on one **or** more **input** features

* The goal of regression **is** to find the relationship between independent variables (features) **and** a dependent variable (target)

Example:

Predicting house price

predicting employee salaries

why regression?

in real world business problems, organizations often need to answer questions such **as**

1. Linear Regression

* Linear Regression **is** the simplest regression algorithm that models the relationship between **input** **and** output variables using a straight line

* It assumes that a linear relationship exists between the independent variable **and** dependent variable

Objective:

* Find the best-fitting straight line that minimizes prediction errors

Equation of linear regression:

for one feature:

$y = mx + b$

where;

y = predicted value

x = **input** feature

m = slope (weight)

b = intercept (bias)

Example:

salary = $5000 * \text{Experience} + 20000$

if experience = **5 years**

salary = $(5000 * 5) + 20000 = 45000$

Why do we need a cost function??

* Initially, the model guesses values **for** slope (m) **and** intercept (b)

* These guesses are usually wrong

* So we need a way to measure:

* "How bad is our prediction?"

* This measurement **is** called the cost function

Mean Squared Error (MSE)

The most common cost function **for** linear Regression **is** MSE.

Formula:

* $MSE = \text{Error}^2 + \text{Error}^2 + \dots + \text{Error}^2 / n$

* Error=Actual Value-Predicted Value

Steps:

1. Find the error for each prediction
2. Square each error
3. Add all squared errors
4. Divide by the total number of observations

example:

Actual	predicted	Errors	Error^2
10	8	2	4
15			
20			

Gradient Descent:

Gradient Descent is a method used to find the best line by reducing the prediction error step by step

Real-life Example:

Imagine you are on top of a mountain and want to reach the lowest point

1. Take a small step downward
2. Check if you're getting lower
3. Keep taking small steps down
4. Eventually, you reach the bottom

* This is exactly how Gradient Descent Works.

In machine Learning

- * Start with random values for slope and intercept
- * Make predictions
- * Calculate the error using cost function
- * Adjust the values slightly to reduce the error
- * Repeat until the error becomes very small

Cell In[1], line 30

```
if experience=5years
```

^

SyntaxError: invalid decimal literal

```
import numpy as np
#Dataset
x=np.array([1,2,3,4])
y=np.array([3,5,7,9])
#Initial values
m=0 #start with random guesses
b=0
learning_rate=0.01 #controls how much the model updates each step
epochs=1000 #Number of iterations
```

```

n=len(x)
for i in range(epochs):
    #predictions
    y_pred=m*x+b          #Current line equation
    #Gradients
    dm=(-2/n)*np.sum(x*(y-y_pred))    #how much to change slope
    db=(-2/n)*np.sum(y-y_pred)        #how much to change
    intercept
    #update parameters
    m=m-learning_rate *dm
    b=b-learning_rate*db              # Move toward lower error
print("Slope(m):",m)
print("Intercept(b):",b)

```

```

Slope(m): 2.0048610156782556
Intercept(b): 0.9857080211211781

```

OLS(Ordinary Least Squares)

- * When we draw a regression line, there can be many possible lines
- * OLS helps us find the best line

How??

1. Predict values using a line
2. Compare predicted values with actual values
3. Find the error
4. Square all errors
5. Add them together
6. choose the line with the smallest total error.

```

import pandas as pd
from sklearn.linear_model import LinearRegression
#Sample Data
df=pd.DataFrame({
    "Experience":[1,2,3,4,5],
    "Salary":[30000,35000,40000,45000,50000]})
#Input Feature (X)
x=df[["Experience"]]
#Target Variable(y)
y=df["Salary"]
#Create Model
model=LinearRegression()
#Train Model(OLS happens here)
model.fit(x,y)
#predict Salary
predictions=model.predict(x)
#Results
print("Slope:",model.coef_[0])
print("Intercept:",model.intercept_)

```

```

print("Predicted Salaries", predictions)
#When fit() runs, OLS automatically calculates the best slope and

Slope: 5000.000000000001
Intercept: 24999.999999999996
Predicted Salaries [30000. 35000. 40000. 45000. 50000.]

new_experience=pd.DataFrame({"Experience":[6,7,8,9]})
#Predict salary for new values
new_predictions=model.predict(new_experience)
print("Predicted Salaries for 6,7,8,9:")
print(new_predictions)

Predicted Salaries for 6,7,8,9:
[55000. 60000. 65000. 70000.]

import matplotlib.pyplot as plt
plt.figure(figsize=(8,5))
# Original data points
plt.scatter(df['Experience'], df['Salary'], color='blue',
label='Actual Data')
# Regression line (trained data)
plt.plot(df['Experience'], predictions, color='red', label='Regression
Line')
# New predictions
plt.scatter(new_experience['Experience'], new_predictions,
color='green')
# Labels
plt.xlabel("Experience (Years)")
plt.ylabel("Salary")
plt.title("Linear Regression: Experience vs Salary")
plt.legend()
plt.grid()
plt.show()

```



Multiple Regression

Multiple Regression **is** a regression technique that uses two **or** more independent variables to predict a dependent variable.

Example:

Predicting a house price using:

1. Area
2. Number of Bedrooms
3. house Age

* Instead of one feature, we use multiple features

Example Dataset

Area	Bedrooms	Age	Price
1000	2	10	300000
1200	3	8	350000
1500	3	5	450000
1800	4	4	500000
2000	4	2	600000

##Multiple Regression Code

```
import pandas as pd
from sklearn.linear_model import LinearRegression
# Dataset
df = pd.DataFrame({
    'Area': [1000, 1200, 1500, 1800, 2000],
    'Bedrooms': [2, 3, 3, 4, 4],
    'Age': [10, 8, 5, 4, 2],
    'Price': [300000, 350000, 450000, 500000, 600000]
})
```

```

# Features
X = df[['Area', 'Bedrooms', 'Age']]    #Using multiple features.
# Target
y = df['Price']
# Model
model = LinearRegression()
# Train
model.fit(X,y)    #Learns the relationship between all features and
price.
# Prediction
pred = model.predict(X)    #predict house price
print("Coefficients:", model.coef_)
print("Intercept:", model.intercept_)
# new data to test
new = pd.DataFrame({
    'Area': [2500, 5448, 7000, 4999],
    'Bedrooms': [2, 1, 4, 5],
    'Age': [1, 45, 34, 23]})

new_pred=model.predict(new)
print(new_pred)

Coefficients: [    250.          -34615.38461538 -13461.53846154]
Intercept: 253846.15384615381
[ 796153.84615385  975461.53846154 1407692.30769231 1020903.84615385]

```

What is Multicollinearity

Multicollinearity occurs when two or more independent variables are highly correlated with each other

Example:

Area sqft	Area_sqm
1000	92.9
1200	111.5

Problem

When features are highly correlated:

1. Model gets confused
2. Coefficients become unstable
3. Interpretation becomes difficult

How to Detect Multicollinearity??

Using:

VIF(Variance Inflation Factor)

VIF measures how much a feature is correlated with other features

VIF Interpretation

VIF	Meaning
1	No correlation
1-5	

>5 Moderate multicollinearity
>10 Severe multicollinearity

#VIF Code

```
import pandas as pd
from statsmodels.stats.outliers_influence import
variance_inflation_factor
# Features
X = df[['Area', 'Bedrooms', 'Age']]
# VIF Calculation
vif = pd.DataFrame()
vif['Feature'] = X.columns
vif['VIF'] = [
    variance_inflation_factor(X.values, i)
    for i in range(X.shape[1])
]
print(vif)
```

	Feature	VIF
0	Area	165.122797
1	Bedrooms	174.483505
2	Age	2.717105

Polynomial Regression

Polynomial Regression **is** an extension of linear Regression that captures non-linear relationships by adding polynomial terms

Why Polynomial Regression??

Not **all** relationships are straight lines

Examples:

Age vs Income

temperature vs Electricity consumption

Degree Selection:

Degree determines model complexity:

* Low degree

1. Simple Model

2. May underfit

* High Degree

1. More flexible

2. May overfit

* Optimal degree

1. Best Balance

2. Good generalization on new data

Advantages:

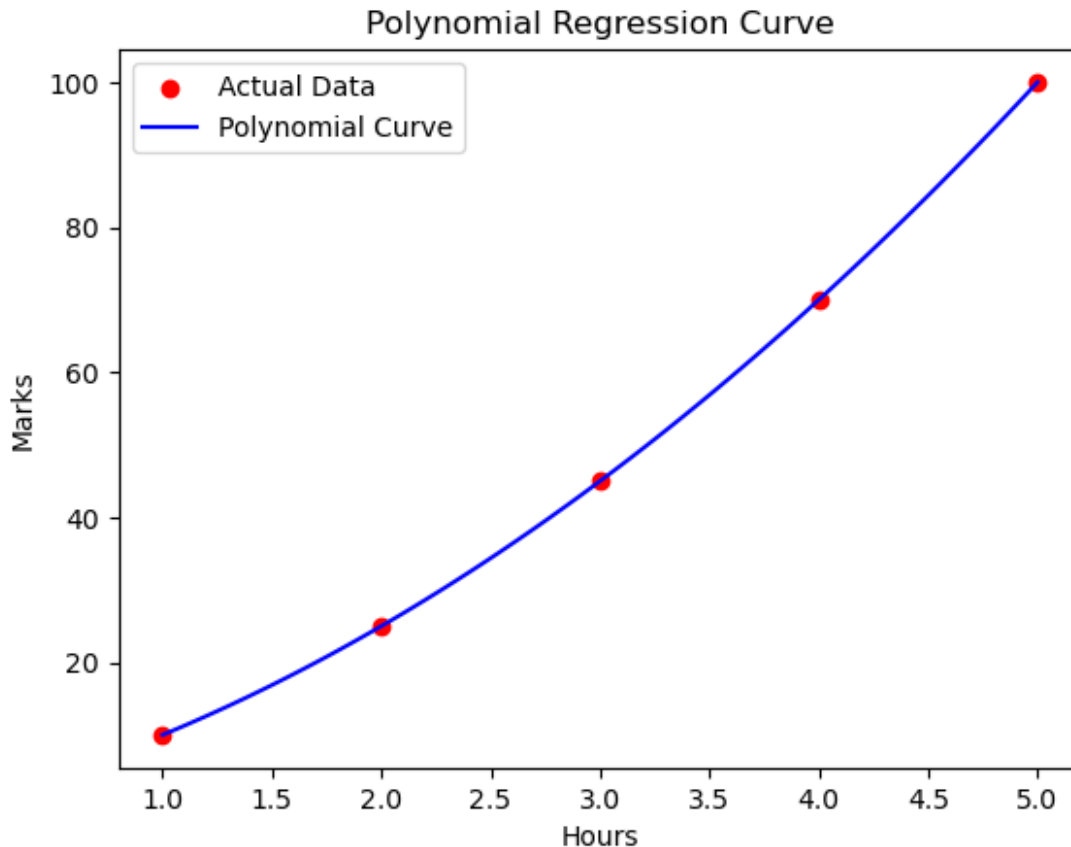
Captures non-linear patterns

More flexible than linear regression

Disadvantages:

Sensitive to outliers
higher risk of overfitting
harder to interpret

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
# Sample dataset
df = pd.DataFrame({
    'Hours': [1, 2, 3, 4, 5],
    'Marks': [10, 25, 45, 70, 100]
})
X = df[['Hours']]
y = df['Marks']
# Polynomial transformation
poly = PolynomialFeatures(degree=4)
X_poly = poly.fit_transform(X)
# Model training
model = LinearRegression()
model.fit(X_poly, y)
# Smooth curve generation (IMPORTANT STEP)
X_range = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)
X_range = pd.DataFrame(X_range, columns=['Hours'])
X_range_poly = poly.transform(X_range)
# Predictions for curve
y_curve = model.predict(X_range_poly)
# Plotting
plt.scatter(X, y, color='red', label='Actual Data')
plt.plot(X_range, y_curve, color='blue', label='Polynomial Curve')
plt.title('Polynomial Regression Curve')
plt.xlabel('Hours')
plt.ylabel('Marks')
plt.legend()
plt.show()
```



Underfitting

occurs when the model *is* too simple

characteristics:

Poor training performance

Poor

Trying to fit a straight line to a curved dataset

Good fit:

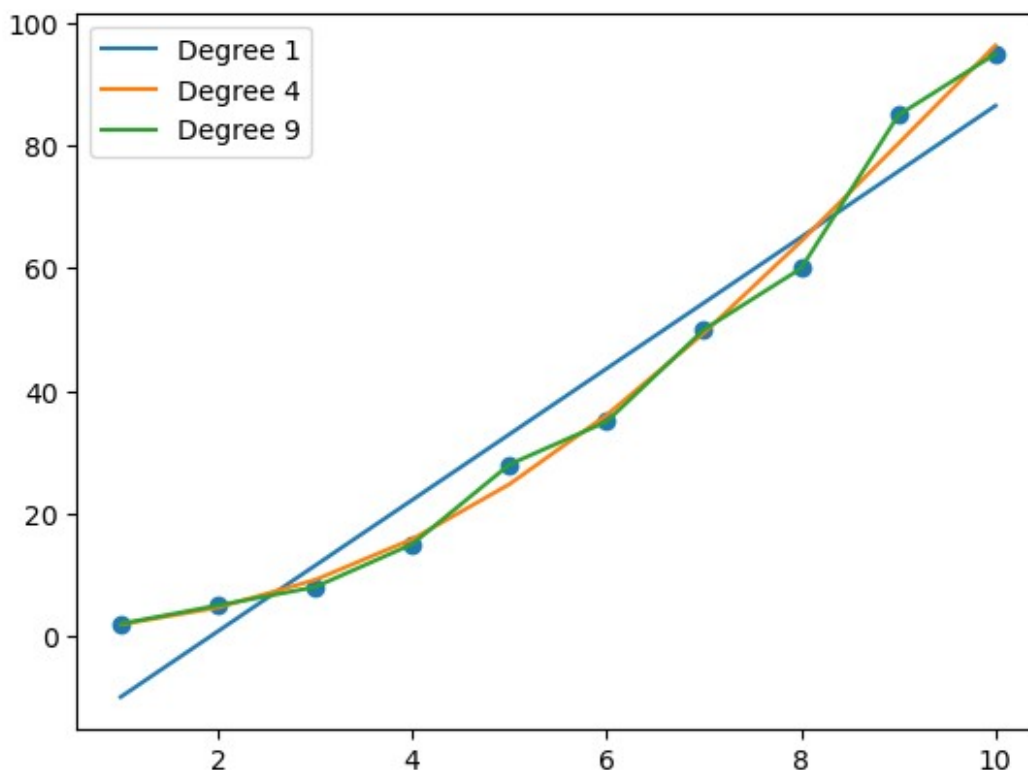
The model captures the underlying pattern

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
X = np.array([1,2,3,4,5,6,7,8,9,10]).reshape(-1,1)
y = np.array([2,5,8,15,28,35,50,60,85,95])
# Degree 1
poly1 = PolynomialFeatures(degree=1)
X1 = poly1.fit_transform(X)
# Degree 4
poly4 = PolynomialFeatures(degree=4)
X4 = poly4.fit_transform(X)
# Degree 9
```

```

poly9 = PolynomialFeatures(degree=9)
X9 = poly9.fit_transform(X)
model1 = LinearRegression().fit(X1,y)
model2 = LinearRegression().fit(X4,y)
model9 = LinearRegression().fit(X9,y)
plt.scatter(X,y)
plt.plot(X, model1.predict(X1), label="Degree 1")
plt.plot(X, model2.predict(X4), label="Degree 4")
plt.plot(X, model9.predict(X9), label="Degree 9")
plt.legend()
plt.show()

```



1 ->underfit --Straight line misses the pattern
 4 ->Good fit --Captures the overall curve
 9 -> overfit --Tries to pass through every point - learns noise instead of pattern

Regularization--- prevents from overfitting

When a machine learning model becomes too complex, it may memorize the training data instead of learning the underlying patterns. This problem is known as overfitting. Regularization is a technique used to reduce overfitting by adding a penalty term to the model's

cost function. The penalty discourages the model from assigning very large values to coefficient, helping

it generalize better to unseen data.

1.Ridge Regression(L2 Regularization)

Ridge Regression adds the sum of squared coefficients as a penalty term to the cost function

Characteristics:

Shrinks coefficient values toward zero

Doesnot make coefficients exactly zero

useful when all features are important

Handles multicollinearity effectively

2.Lasso Regression(L1 Regularization)

Lasso Regression adds the sum of absolute coefficient values as a penalty term

characteristics:

Shrinks coefficients toward zero

Can make some coefficients exactly zero

performs automatic feature selection

useful when many irrelevant features exist

elastic net Regression

ElasticNet combines both Ridge(L2) and lasso(L1) penalties

Characteristics:

combines advantages of ridge and lasso

Performs feature selection

handles multicollinearity

suitable for datasets with many correlated features

When to use each method

Method	When to use
Ridge(L2)	When all features are important and multicollinearity exists
Lasso(L1)	When feature selection is required
ElasticNet	When features are highly correlated and feature selection is also needed

##Ridge Code

```
from sklearn.linear_model import Ridge
from sklearn.datasets import make_regression
X, y = make_regression(
    n_samples=100,
    n_features=5,
    noise=20,
    random_state=42
```

```
)
ridge = Ridge(alpha=1.0)
ridge.fit(X, y)
print("Coefficients:")
print(ridge.coef_)

Coefficients:
[62.4844981  98.07649112 57.01070157 52.31272567 35.35343002]
```

##Lasso Code

```
from sklearn.linear_model import Lasso
from sklearn.datasets import make_regression
X, y = make_regression(
    n_samples=100,
    n_features=10,
    noise=20,
    random_state=42
)
lasso = Lasso(alpha=2.0)
lasso.fit(X, y)
print("Coefficients:")
print(lasso.coef_)
```

```
Coefficients:
[17.88753444 54.76076704 0.          60.9632585  90.0800089
 68.30461063
 81.35787563  4.03519677  2.86610797 67.99906953]
```

##Lasso Code

```
from sklearn.linear_model import ElasticNet
from sklearn.datasets import make_regression
X, y = make_regression(
    n_samples=100,
    n_features=10,
    noise=20,
    random_state=42
)
ElasticNet = ElasticNet(alpha=3.0, l1_ratio=1.0)
ElasticNet.fit(X, y)
print("Coefficients:")
print(ElasticNet.coef_)
```

```
Coefficients:
[16.80614975 53.82143974 0.          60.29629184 88.77702333
 67.6125381
 80.18743145  2.46091457  2.267352  66.71648422]
```

Regression Matrix-MSe,RMSE,MAE,R2,Adjusted R2
why do we need regression metrics?

1. MSE(Mean Squared Error):

MSE is the average of squared difference between actual and predicted values

Explanation:

calculated error=Actual-Predicted

Square it

2. RMSE(Root Mean Squared Error)

RMSE is the square root of MSE

Formula ----> $RMSE = \sqrt{MSE}$

Explanation:

same as MSE but converted back to original units

Easier to understand than MSE

Why RMSE?

MSE is in squared units, which is hard to interpret

Interpretation:

Lower RMSE -> better performance

Represents "average error" in real units

2. MAE(

MAE is the average of absolute differences between actual and predicted values

key idea:

All errors are treated equally

Difference from MSE:

MAE ----> linear penalty

MSE ----> penalizes large errors more

Interpretation:

Lower MAE --> better model

4. R2(R square) Score(coefficient of Determination)

R2 tells how much variance in data is explained by the model

Formula ---> $R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$

Explanation:

Measures how well the model fits the data

Think of it as "accuracy percentage" for regression

1 --> Perfect Model

0 --> Very poor model

Negative --> Worse than average model

Interpretation:

Higher R2 --> Better model

5. Adjusted R2:

Problem with R2:

R2 always increases when we add more features, even if they are useless

Solution: Adjusted R2

Adjusted R2 fixes this by penalizing unnecessary features

Formula: $\text{Adjusted R2} = 1 - \frac{(1 - R^2) \cdot n}{n - p - 1}$

where:

n = number of samples

p = number of features

Explanation:

Only increase if new feature

Mini summary Table

Metric	What is measures	Best value
MSE	Average squared error	Lower
RMSE	Average Error(real units)	Lower
MAE		Lower
R2		Higher
Adjusted R2		Higher